

ToddySoft Connect S7 Driver - Ignition

8.1

Ignition 8.1 module for connecting to Siemens SIMATIC controllers — S7-300, S7-400, S7-1200, S7-1500 and family devices such as Simocode, Sinamics and LOGO! — over ISO-on-TCP. The driver speaks both classic S7Comm and S7CommPlus, the TLS-secured native protocol of the S7-1200 / S7-1500, and negotiates between them at connect time.

Choosing between this module and the S7 Web API module. This one is the process-data path: binary framing, batched reads, real subscriptions. The **S7 Web API** module reaches surfaces this driver cannot — the diagnostic buffer, alarms, the operating mode — over HTTPS, at roughly a tenth of the per-tag speed and with no change notification on the wire. Use this module to collect data, and the Web API module for diagnostics or where network policy permits only HTTPS.

Installation

This module is for Ignition 8.1 only. It is built against the Ignition 8.1 SDK and will not run on any other major version. For Ignition 8.3, install the separate `...-8.3.mod1` build instead — the two are not interchangeable, and each gateway needs the build that matches it.

Ignition 8.0 and earlier are not supported and cannot be. The module is built on the device-driver API that Ignition introduced with 8.1, so on 8.0 the gateway fails to load it with `NoClassDefFoundError: ... opcu.server.api.AbstractDeviceModuleHook`.

Both builds carry the same name in the gateway's module list, so tell them apart by the file you installed or by the module description, which names the Ignition version.

The ToddySoft Connect modules are signed with a self-signed certificate (issued to *ToddySoft GmbH*) rather than a publicly accredited code-signing certificate. Ignition therefore asks you to review and accept that certificate once during installation - it is then remembered for all future ToddySoft modules signed with the same certificate.

1. Open the Ignition Gateway web interface and navigate to **Config > System > Modules**. Scroll to the bottom and click **Install or Upgrade a Module...**
2. Choose the `ToddySoft-Connect-S7-Driver-8.1.modl` file and click **Install**.
3. Ignition shows the self-signed certificate details (subject and issuer *ToddySoft GmbH*) together with a warning that the module was not written by Inductive Automation. Review the certificate, tick **I understand the risk and want to install this module** and click **Add Certificate and Install Module**.
4. The module is now trusted and listed under the **ToddySoft GmbH (self-signed certificate)** group with the state **Running**.

ToddySoft GmbH (self-signed certificate) View Certificate					
Name	Version	Description	License	State	
ToddySoft Connect S7 Driver	1.3.0-SNAPSHOT (b0)	ToddySoft Ignition driver for Siemens S7 protocol. Built for Ignition 8.1.	Trial	Running	More ▾ restart
ToddySoft Connect S7 Web API Driver	1.3.0-SNAPSHOT (b0)	ToddySoft Ignition driver for the Siemens SIMATIC Web API. A management-plane path for diagnostics, alarms and operating mode; use the S7 module for process data. Built for Ignition 8.1.	Trial	Running	More ▾ restart

[→ Install or Upgrade a Module...](#)

Creating a Device Connection

1. Go to **Config > OPC UA > Device Connections**.

Config > OPCua > Devices Trial Mode 1:58:22 We're glad you're test driving our software. Have fun. Activate Ignition				
Name	Type	Description	Enabled	Status
No Devices				
→ Create new Device...				

2. Click **Create new Device...**, select **ToddySoft Connect S7 Driver** as the device type and click **Next**.

☐ **Siemens S7-1500**
Connect to Siemens S7-1500 PLCs over Ethernet.

☐ **Siemens S7-300**
Connect to Siemens S7-300 PLCs over Ethernet.

☐ **Siemens S7-400**
Connect to Siemens S7-400 PLCs over Ethernet.

☐ **TCP Driver**

☒ **ToddySoft Connect S7 Driver**
ToddySoft Connect driver for Siemens S7 protocol

☐ **ToddySoft Connect S7 Web API Driver**
ToddySoft Connect driver for the Siemens SIMATIC Web API (HTTPS) on S7-1200 and S7-1500. A management-plane path for diagnostics, alarms and operating mode - roughly an order of magnitude slower per tag than the S7 driver, with no change notification on the wire. Use the S7 driver for process data.

☐ **UDP Driver**

3. Configure the connection settings (see the sections below) and click **Save**.

4. The new device then appears in the Device Connections list.

✓ Successfully created new Device "Sorting-Lane-S7"

Name	Type	Description	Enabled	Status	
Sorting-Lane-S7	ToddySoft Connect S7 Driver	S7-1200 on the sorting lane	true	Connected	<button>delete</button> <button>edit</button>

[→ Create new Device...](#)

Device discovery is an Ignition 8.3 feature. It needs the custom configuration form the 8.3 module ships, which the 8.1 device-settings API cannot render. On 8.1, enter the host, rack and slot by hand — everything else in this manual applies to both versions. Neither version offers a *Test Connection* action for this driver.

Setting blocks

The configuration is organized into categories: **General**, **Transport Settings**, **S7 Settings**, **Audit Log Settings**, **Limits**, **TLS Settings** and **Authentication**.

General

Assigns a name (and optional description) to the connection and controls whether it is enabled.

General

Name

Sorting-Lane-S7

Description

S7-1200 on the sorting lane

Enabled

☒ (default: true)

Setting	Description	Default
Name	Unique name for this device connection	
Description	Optional description for this connection	
Enabled	Whether the connection is active	true

This is the name the connection is then referenced under in other parts of Ignition.

If `Enabled` is not checked, no connection is established until it is enabled.

Transport Settings

Transport Settings	
Host	192.168.23.30 Hostname or IP address of the S7 PLC
Port	102 TCP port number (default: 102) (default: 102)

Setting	Description	Default
Host	Hostname or IP address of the S7 PLC	
Port	TCP port number	102

There is no transport selector on this driver, and none is missing. S7 speaks exactly one transport — `cotp`, ISO-on-TCP per RFC 1006 — and **TLS here is not a transport**: the S7CommPlus handshake runs *inside* COTP framing on the same port 102. There is no second port and no separate scheme to choose. Whether the session ends up encrypted is decided by the **TLS Settings** block below, not here.

S7 Settings

S7 Settings	
Rack	0 Rack number of the S7 PLC (default: 0) (default: 0)
Slot	1 Slot number of the S7 CPU (default: 1) (default: 1)
Read Timeout	5000 Maximum time to wait for data on the connection before it is treated as lost, in milliseconds. Maps to the driver's read-timeout-ms; the driver's own default is 1000 (default here: 5000) (default: 5,000)
Browse Queries	** Query string for symbol browsing (default: ""). Separate multiple queries with ";" (default: "")

Setting	Description	Default
Rack	Rack number of the S7 PLC	0
Slot	Slot number of the S7 CPU	1
Read Timeout	Maximum time to wait for data on the connection before it is treated as lost, in milliseconds	5000
Browse Queries	Symbol browse query filter. Separate multiple queries with ;	**

Rack and slot are worth double-checking. They identify *which CPU* in the rack the session addresses, and a wrong pair frequently connects — to a different CPU, which then answers reads with perfectly valid data from the wrong machine. The usual values

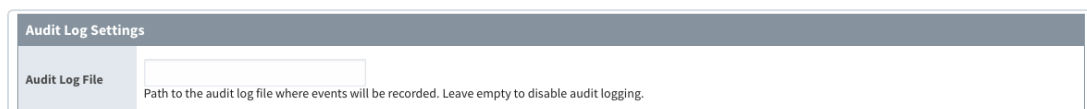
are rack 0 / slot 1 for an S7-1200 or S7-1500, and rack 0 / slot 2 for a classic S7-300 / S7-400.

Read Timeout is how long the driver waits for data on an established connection before treating the link as lost — the S7 protocol has no per-request budget, and this is the nearest concept it offers. A slow controller under load does not need this raised; a flaky network link does.

(Before 1.3.0 this field was labelled *Request Timeout* on Ignition 8.1, which named a setting the protocol does not have. Only the label changed; the value you had configured is unaffected.)

Audit Log Settings

The **Audit Log** writes important diagnostic information to a dedicated log file. This file can grow quickly, so it is advised to only enable it when investigating issues.



Setting	Description	Default
Audit Log File	Absolute path to the audit log file where events will be recorded. Leave empty to disable.	(empty)

NOTE: Please ensure the user the **Ignition** process is running under has write access to this path.

The Audit Log system automatically creates the file and rotates it (creating compressed files next to it) once a certain size is reached or on a time-based schedule.

Limits

Ceilings on what the controller can make the driver allocate. Each one bounds something the *other* side chooses, so a device that misbehaves — or a malformed frame on the wire — cannot exhaust the gateway. The defaults are the driver's own; raise one only for a controller measured to need it. Leaving them alone produces exactly the connection the device made before these settings existed.

Limits	
Max Object Depth	20 Maximum nesting depth accepted when parsing S7CommPlus objects. An object's children array is terminated by a byte marker, so without a limit the nesting depth is decided by the controller rather than by the driver. Real responses are a handful of levels deep (default: 20)
Max VLAN Depth	8 Maximum 802.1Q VLAN nesting depth accepted when parsing a captured Ethernet frame during discovery. Discovery frames arrive as unauthenticated broadcast, at four bytes per level. The default of 8 admits 7 stacked tags; standard Q-in-Q is two (default: 8)
Max Tag Bytes	8388608 Maximum bytes a single tag address may command the driver to allocate. An address carries its own element count, and that count sizes client-side buffers before any I/O reaches the controller, so a handful of oversized addresses can exhaust a collector without the PLC being involved at all (default: 8388608)

Setting	Description	Default
Max Object Depth	Maximum nesting depth accepted when parsing S7CommPlus objects. An object's children array is terminated by a byte marker, so without a limit the nesting depth is decided by the controller rather than by the driver. Real responses are a handful of levels deep.	20
Max VLAN Depth	Maximum 802.1Q VLAN nesting depth accepted when parsing a captured Ethernet frame during discovery. Discovery frames arrive as unauthenticated broadcast, at four bytes per level. The default of 8 admits 7 stacked tags; standard Q-in-Q is two.	8
Max Tag Bytes	Maximum bytes a single tag address may command the driver to allocate. An address carries its own element count, and that count sizes client-side buffers before any I/O reaches the controller.	8388608

TLS Settings

S7CommPlus can run the session inside TLS, and on an S7-1200 / S7-1500 it normally does. These settings decide whether that happens and what the driver accepts on the other end.

TLS Settings	
Minimum Protocol	CLASSIC The lowest protocol this connection may end up using. The driver negotiates downwards, and a missing or mangled handshake response drops the session to plaintext S7Comm. Set this to V3 to make that downgrade a connect error instead (default: Classic, which permits it and logs a warning) (default: CLASSIC)
Maximum Protocol	V3_TLS The highest protocol this connection may attempt. Set this to Classic to pin the connection to legacy S7Comm and skip S7CommPlus negotiation entirely, for a controller that misbehaves on the modern protocol (default: V3) (default: V3_TLS)
Verify Certificate	☒ Validate the controller's TLS certificate. Turning this off accepts any certificate and gives encryption without authentication - development only. Modern S7-1500 firmware ships a self-signed device certificate, so either pin its fingerprint or supply a trust store (default: on) (default: true)
Pinned Certificate Fingerprints	Comma-separated SHA-256 fingerprints of the leaf certificates to accept (hex, colons optional). When set, only these certificates are accepted and Verify Certificate is overridden. This is the supported way to reach a controller with a factory self-signed certificate; the driver logs the controller's fingerprint on every connection, ready to copy
Trust Store Path	Path to a JKS or PKCS12 trust store holding the CA that signed the controller certificate. Used in addition to the Java runtime's own trust anchors. Leave empty to rely on those alone
Trust Store Password	Password for the trust store. Leave empty for an unprotected store
TLS Version	TLSv1.3 Comma-separated allow-list of TLS versions to offer. Add TLSv1.2 for an older controller that does not support 1.3 (default: TLSv1.3) (default: TLSv1.3)

Setting	Description	Default
Minimum Protocol	The lowest protocol this connection may end up using	CLASSIC
Maximum Protocol	The highest protocol this connection may attempt	V3_TLS
Verify Certificate	Validate the controller's TLS certificate	true
Pinned Certificate Fingerprints	Comma-separated SHA-256 fingerprints of the leaf certificates to accept (hex, colons optional)	(empty)
Trust Store Path	JKS or PKCS12 trust store holding the CA that signed the controller certificate	(empty)
Trust Store Password	Password for the trust store. Leave empty for an unprotected store	(empty)
TLS Version	Comma-separated allow-list of TLS versions to offer	TLSv1.3

The protocol window, and the silent downgrade

The two dropdowns list the values as **CLASSIC** and **V3_TLS** on Ignition 8.1.

The Ignition 8.3 module spells the same two options out as *Classic (plaintext S7Comm)* and *V3 (S7CommPlus over TLS)*; the setting and its effect are identical, only the wording in the list differs.

The driver negotiates **downwards**. If the controller's handshake response is missing or mangled, the session drops to plaintext S7Comm and carries on working — the tags read, the device shows *Connected*, and nothing announces that the encryption is gone. On a hostile network segment, an attacker who drops a single response downgrades the session to cleartext.

That default exists because it is the behaviour that connects to the widest range of hardware, and the downgrade is logged at `WARN`. **On any connection you consider security-relevant, set *Minimum Protocol* to `V3_TLS`**. The downgrade then becomes a connect error instead of a warning nobody reads.

Maximum Protocol works the other way: setting it to `CLASSIC` pins the connection to legacy S7Comm and skips S7CommPlus negotiation entirely. That is the workaround for a controller that misbehaves on the modern protocol, not a setting to reach for casually.

Trusting the controller's certificate

Modern S7-1500 firmware ships a **self-signed device certificate**. It is not signed by any CA, so ordinary verification cannot succeed against it, and there are exactly two honest ways forward:

1. **Pin the fingerprint** (recommended). Paste the controller's SHA-256 fingerprint into **Pinned Certificate Fingerprints**. When set, only those certificates are accepted and *Verify Certificate* is overridden. The driver logs the controller's fingerprint on every connection, so you never have to compute it by hand. This is secure *and* it works out of the box.
2. **Supply a trust store**, if the certificate was signed by your own CA. The store is used in addition to the Java runtime's own trust anchors.

Turning **Verify Certificate** off gives you encryption without authentication — anyone who can intercept the connection can terminate the TLS themselves. It is a development setting. Pin a fingerprint instead; it is no more work.

TLS Version defaults to `TLSv1.3`. Add `TLSv1.2` for an older controller that does not support 1.3 — the field is an allow-list, so `TLSv1.3, TLSv1.2` offers both.

Authentication

S7CommPlus **legitimation** — the driver's only authentication method, and a protocol-layer one. It runs inside the S7CommPlus conversation once the connection is up, so it is unrelated to the TLS settings above: a plaintext session can be legitimated and an encrypted one need not be.

Authentication	
Username	 Account name on a user-managed controller, such as an S7-1200 G2. Leave empty for the legacy arm, which authenticates with a device password alone
Password	 Controller password for S7CommPlus legitimation. Leave empty for a controller that does not require it. Stored in encoded form and redacted from all log output

Setting	Description	Default
Username	Account name on a user-managed controller, such as an S7-1200 G2. Leave empty for the legacy arm, which authenticates with a device password alone	(empty)
Password	Controller password. Leave empty for a controller that does not require legitimation	(empty)

Whether you need this at all depends on the controller's **protection level**, which is set in the TIA project and is not knowable until the driver connects. A controller that does not demand legitimation connects with no credential, which is the common case — leave both fields alone.

The password is what authenticates. A username alone legitimates nothing (both arms present a password), so nothing is sent unless a password is set. There is no arm selector: the driver picks between the legacy and named-user mechanisms from the controller's article number and firmware, and supplying a username is itself enough to select the named-user arm.

The password is stored in encoded form, and since ToddySoft Connect 1.2.0 it is handed to the driver as an object rather than written into the connection string — so it does not appear in the gateway log, in a support bundle, or in any string quoting the connection.

Tag Addressing

This driver accepts two address forms. They are not alternatives to pick between: the absolute form works everywhere, the symbolic form works only on S7-1200 / S7-1500 via S7CommPlus.

Absolute addresses (all controllers)

Siemens' own absolute-address style:

```
%<area><offset>[,<bit>]:<datatype>[(<size>)] [[<selection>]]
area      ::= DB<n> | M | I | Q | T | C
```

Address	Meaning
%DB1:0:INT	Signed 16-bit int at byte 0 of DB1

Address	Meaning
<code>%DB1:4.0:B00L</code>	Bit 0 of byte 4 of DB1
<code>%DB1:10:STRING(20)</code>	S7 STRING (length-prefixed, up to 20 chars) at byte 10 of DB1
<code>%DB1:0[0..9]:REAL</code>	Ten floats starting at byte 0 of DB1 — a range, inclusive of both bounds
<code>%DB1:0[10]:REAL</code>	One float, the eleventh (index 10). A bare index is a single element, not a count
<code>%DB1:0[4..7;1]:REAL</code>	Elements 4–7 of an array TIA declares as <code>Array[1..10]</code>
<code>%DB1:0[0..1][0..2]:INT</code>	A 2×3 block of ints. One bracket per dimension; <code>[0..1,0..2]</code> is the same selection
<code>%M0.0:B00L</code>	Bit 0 of M0 (merker bit)
<code>%I0:BYTE</code>	Input byte 0
<code>%Q0.5:B00L</code>	Bit 5 of Q0 (output bit)

The longer `%DB<n>.DBX<byte>.<bit>:<datatype>` form is also accepted. Addresses typically come from the TIA Portal symbol export.

An array selection is written *before* the `:TYPE` suffix —

`%DB1:0[0..9]:REAL`, not `%DB1:0:REAL[0..9]`. The old position no longer parses, and the error names the replacement. The parenthesis is reserved for a string's declared length and means nothing else.

Symbolic addresses (S7-1200 / S7-1500)

On an S7-1200 or S7-1500 a tag can be addressed by name, in either of two equivalent forms:

Form	Example	Notes
Block name, quoted	<code>"MyDataBlock".g_plant.channels[2].setpoints[3]</code>	TIA Portal's own display form — copy-pasteable straight out of TIA. This is what browsing produces.
Block number	<code>DB43.g_plant.channels[2].setpoints[3]</code>	Always valid. Used when a block's name is missing or not unique.

Both resolve to the same tag, and addresses already written against the number form keep working.

Why the block name is quoted. A data block may legitimately be *named* `DB42`, so a bare `MyDataBlock.myVar` could not be told apart from the number form without knowing every block name on the controller. The quotes also let a block name contain spaces or dots without escaping — `"OPC Data".myVar`, `"Plant.Line1".speed`.

An ambiguous name is refused, not guessed. The name comes from the controller's type-name attribute, so two instance data blocks of the same function block can report the same string. Addressing such a name directly fails with a message naming the colliding blocks, rather than resolving to whichever was loaded first and returning wrong values from a read reporting success.

A block or memory-area node is a container, not an address. In the browse tree a data block (`"MyDataBlock" / DB42`) and a memory area (`%M`, `%I`, `%Q`) carry no tag and report neither readable nor writable. Read its children, not the node itself. `%DB42:0:BYTE` remains a perfectly valid hand-written address — only the browse tree stopped offering it as a stand-in for a whole block.

Configure **Browse Queries** (e.g. `DB1.*`) to control which symbols appear when browsing tags in the Ignition Designer.

Tag Groups

By default, all tags are collected in the default tag group, which means all selected tags are fetched at the default interval of once a second.

If different intervals are desired, create additional tag groups, configure the desired intervals there, and assign tags to the corresponding tag groups.

Troubleshooting

Device shows "Faulted":

- Verify the PLC hostname/IP is reachable
- Check that **Rack** and **Slot** match the physical configuration — `0 / 1` for S7-1200 and S7-1500, `0 / 2` for a classic S7-300 / S7-400
- Ensure port 102 is not blocked by a firewall
- Review Gateway logs: **Status > Diagnostics > Logs**

The device connects, but the values are from the wrong machine:

- Almost always a wrong **Rack** / **Slot** pair addressing a different CPU in the same rack. It connects, and it reads.

Tags show bad quality:

- Verify the variable exists in the PLC program
- Check spelling and case sensitivity; a quoted block name must match TIA exactly
- Ensure the PLC program is running
- An absolute address written with the array selection after the type — `%DB1:0:REAL [0..9]` — no longer parses; move the selection before the type

Writes are refused although reads work:

- The controller's **protection level**, set in the TIA project, governs writes separately from reads. Supply the controller password under **Authentication**, or lower the protection level in TIA.

The connection is not encrypted although the controller supports TLS:

- The driver negotiates downwards and a missing or mangled handshake response drops the session to plaintext S7Comm, logging a `WARN`. Set **Minimum Protocol** to `V3_TLS` to turn that into a connect error. See *The protocol window, and the silent downgrade*.

TLS connection fails:

- A factory self-signed device certificate cannot pass ordinary verification. Pin its fingerprint under **Pinned Certificate Fingerprints** — the driver logs it on every connection
- For a certificate signed by your own CA, set **Trust Store Path**
- For an older controller, add `TLSv1.2` to **TLS Version**

Docker related issues

Unlike ADS, the S7 protocol carries no identity for this end of the connection, so a container's internal address is never announced to the controller and nothing has to be reconciled. A gateway in a bridged container connects to an S7 controller normally, provided port 102 is routable.

VPN related issues

A remote controller reached over a VPN needs nothing special from this driver, for the same reason: there is no source identity to configure. If the connection fails, it is ordinary reachability — confirm that port 102 is routed and not blocked.

Several devices on the same PLC

Several Ignition devices may address the same controller at once; there is no per-gateway identity to collide, as there is with ADS. What is limited is the controller itself: an S7-300 or S7-400 has a small fixed number of connection resources, and once they are exhausted further connects are refused until one is released. An S7-1200 / S7-1500 has considerably more, but not an unlimited number.

If devices on the same controller connect and drop in rotation, count the connections the controller is serving — including TIA Portal sessions and HMIs — before looking for a fault in the module.

Version history

Unreleased — 1.3.0

Built on ToddySoft Connect 1.3.0.

- **Added:** this manual.
- **Changed:** the **Request Timeout** setting is now labelled **Read Timeout**, and its description says what it actually does. The s7 driver has no request timeout — the value has always been sent as the driver's read timeout, the time it waits for data before treating the link as lost — so the old label named a setting the protocol does not have. Only the label and description changed; the value you had configured is kept and behaves exactly as before. Ignition 8.3 already used the correct label.